

```
<!--Arquitectura de computadoras-->
```

Proyecto
Final {

```
<Por="Null"/>
```

}



Equipo {



DANIELA
GARCÍA

INGENIERA



FÁTIMA
ESPINOZA

INGENIERA



AYLIN
LIZAOLA

INGENIERA



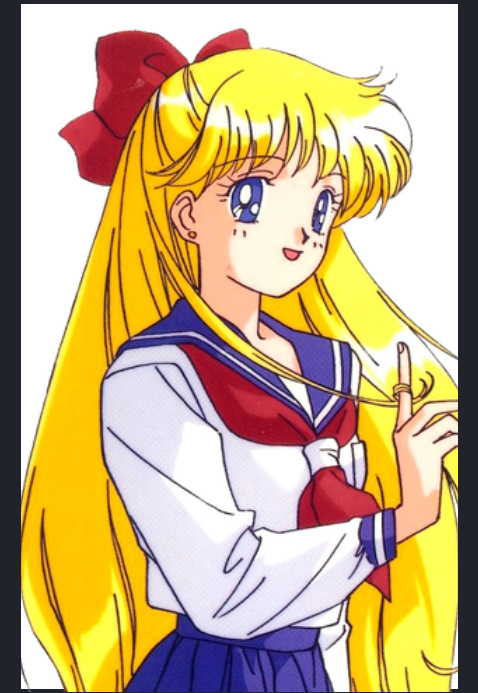
JOSE
RECAMIER

INGENIERO



MARÍA
LOMELÍ

INGENIERA



VALERIA
FERRER

INGENIERA

}

Contenidos

- 01 Introducción
- 02 Fundamentos Teóricos
- 03 Puntos Clave
- 04 Instrucciones Tipo R I J
- 05 Justificación teorica
- 06 Implementación en Verilog
- 07 Modificaciones
- 08 Evidencia Verilog
- 09 Codificador de Python
- 10 Análisis de resultados
- 11 Conclusiones y Referencias

Introducción {

Como parte del proyecto final, se propone desarrollar un programa en Python capaz de traducir instrucciones en ensamblador MIPS tipo R hacía código máquina binario.

El objetivo de este programa es generar instrucciones binarias que posteriormente serán cargadas en la memoria de instrucciones del datapath implementado en Verilog, permitiendo verificar el correcto funcionamiento de cada parte del programa

- Instrucción escrita.
- Separar la operación y los registros.
- Identificará el tipo de instrucción.
- Buscará el código correspondiente.
- Construirá la instrucción de 32 bits.
- Mostrará el resultado en binario.



}

Fundamentos Teóricos {

<Arquitectura MIPS de 32 bits: Diseño, Estructura e ISA/>

Filosofía RISC

MIPS se basa en la computación de conjunto de instrucciones reducido, priorizando:

- Instrucciones fijas: Todas miden 32 bits, simplificando la decodificación.
- Arquitectura Load/Store: Solo las instrucciones lw y sw acceden a memoria.
- Eficiencia: Ejecución ideal de una instrucción por ciclo de reloj.

Jerarquía de Registros

Implementa 32 registros de propósito general (GPR), cada uno de 32 bits de ancho:

- \$zero (0): Valor constante cero por hardware.
- \$s0-\$s7: Registros salvados para variables locales.
- \$ra (31): Dirección de retorno para llamadas a subrutinas (instrucción jal).

}

Puntos clave {

01

Unidad de
control
principal

02

Banco de
registros

05

PC
Módulos de extensión
Multiplexores

03

ALU: Ejecuta la
operación física.
ALU control:
Decodifica las señales
globales de ALUop

04

Gestión de
memorias
(RAM y ROM)

}

Formato de Instrucciones {

Formato de Instrucción Tipo R (Registro)

31	op	25	rs	20	rt	15	rd	10	sh	5	fn	0
	6 bits		5 bits		5 bits		5 bits		5 bits		6 bits	
Opcode		Source register 1		Source register 2		Destination register		Shift amount		Opcode extension		

Formato de Instrucción Tipo I (Inmediato)

31		25		20		15		0	
op		rs		rt		operand / offset			
6 bits		5 bits		5 bits		16 bits			
Opcode		Source or base		Destination or data		Immediate operand or address offset			

Formato de Instrucción Tipo J (Jump)

op		jump target address	
31	25		0
6 bits	26 bits		
Opcode	Memory word address (byte address divided by 4)		

El ensamblador convierte cada instrucción a su representación binaria de 32 bits, que se carga en la memoria de instrucciones mediante un archivo .mem

Campo	Valor
Opcode	000000
rs (\$t0)	01000
rt (\$t1)	01001
rd(\$t2)	01010
shamt	00000
funct (add)	100000

}

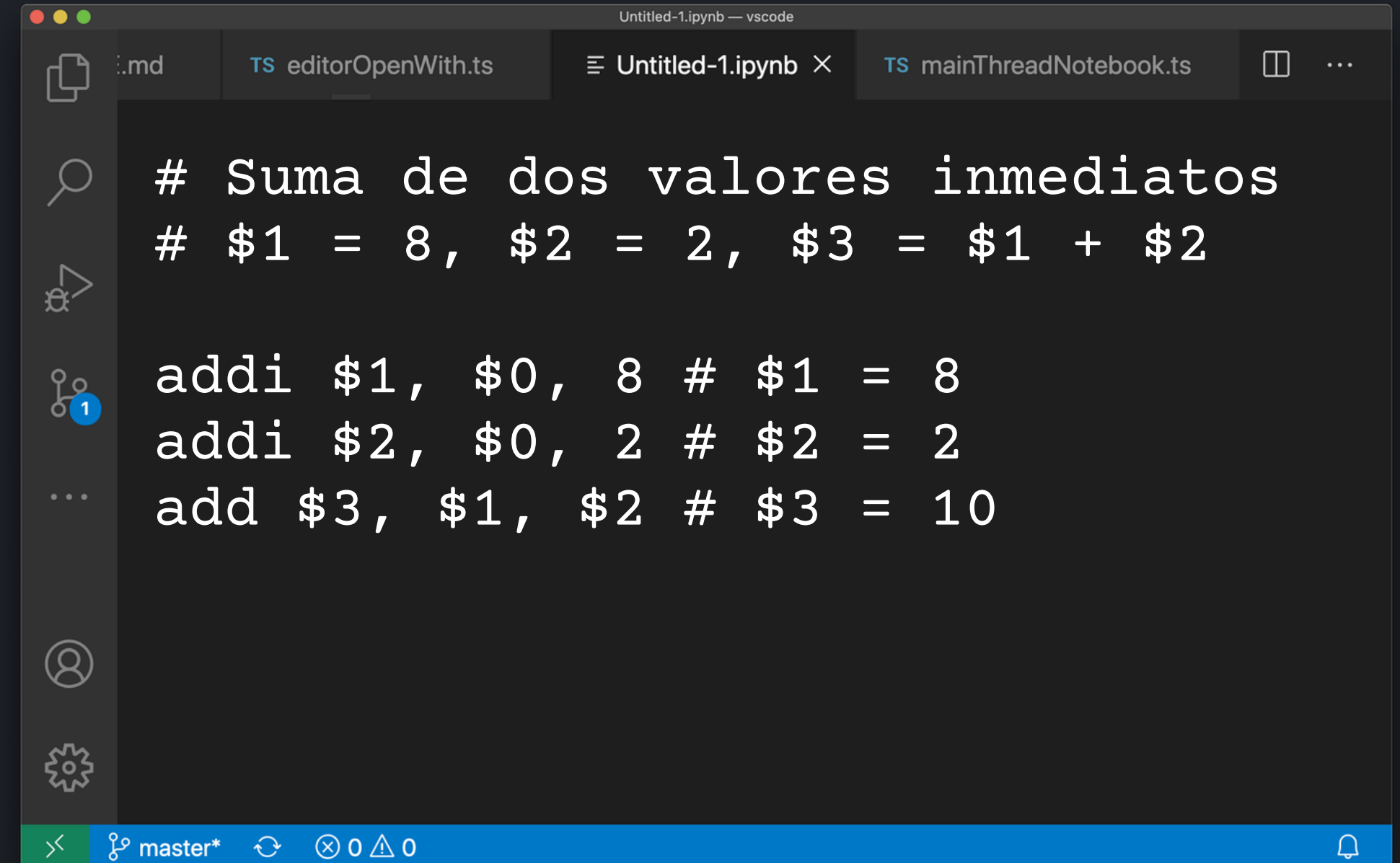
Instrucción Tipo R (Registro) {

Se utilizan para realizar operaciones aritméticas y lógicas (como add, sub, and)

donde todos los operandos provienen de los registros internos del procesador y el resultado se guarda en otro registro.

No interactúan con la memoria de datos ni utilizan números constantes.

Ejemplo en ModelSim



The screenshot shows a VS Code editor window titled 'Untitled-1.ipynb — vscode'. The editor contains the following MIPS assembly code:

```
# Suma de dos valores inmediatos
# $1 = 8, $2 = 2, $3 = $1 + $2

addi $1, $0, 8 # $1 = 8
addi $2, $0, 2 # $2 = 2
add  $3, $1, $2 # $3 = 10
```

The editor interface includes a sidebar on the left with icons for Explorer, Search, Run and Debug, Source Control, and User. The bottom status bar shows 'master*' and other VS Code status indicators.

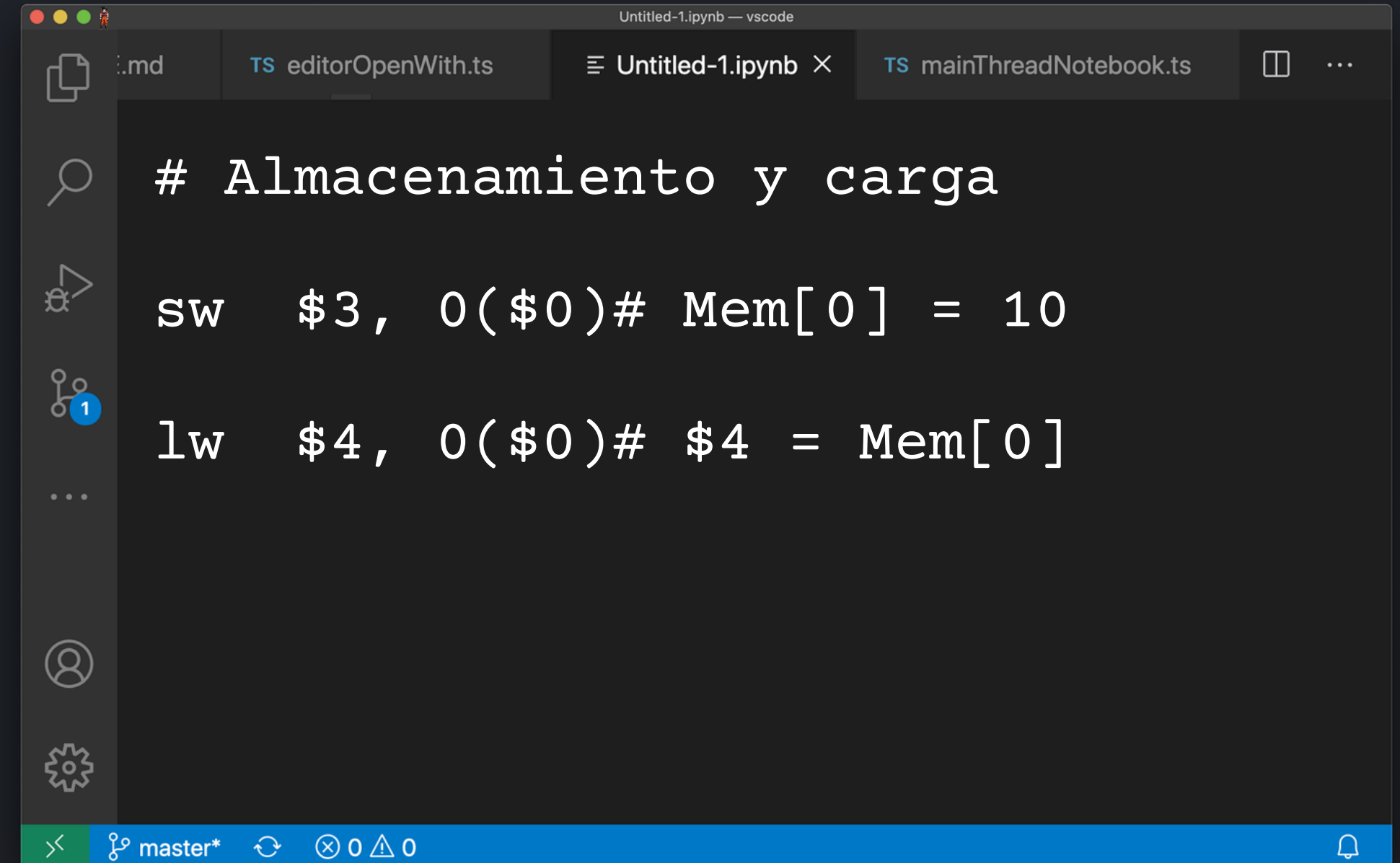
}

Instrucción Tipo I (Inmediato) {

Se emplean cuando una operación necesita trabajar con un valor constante (un número "inmediato" integrado en la propia instrucción) o cuando se necesita leer/escribir en la memoria de datos.

Son vitales para instrucciones de carga/almacenamiento (lw, sw), saltos condicionales (beq) o aritmética con constantes (addi).

Ejemplo en ModelSim



The screenshot shows a VS Code editor window with the title bar "Untitled-1.ipynb — vscode". The editor has several tabs open: ".md", "TS editorOpenWith.ts", "Untitled-1.ipynb X", and "TS mainThreadNotebook.ts". The active tab is ".md", which contains the following MIPS assembly code:

```
# Almacenamiento y carga

sw    $3, 0($0)# Mem[0] = 10

lw    $4, 0($0)# $4 = Mem[0]
```

The left sidebar shows the Explorer view with a file named ".md" and a search icon. The bottom status bar shows "master*" and "0 0 0".

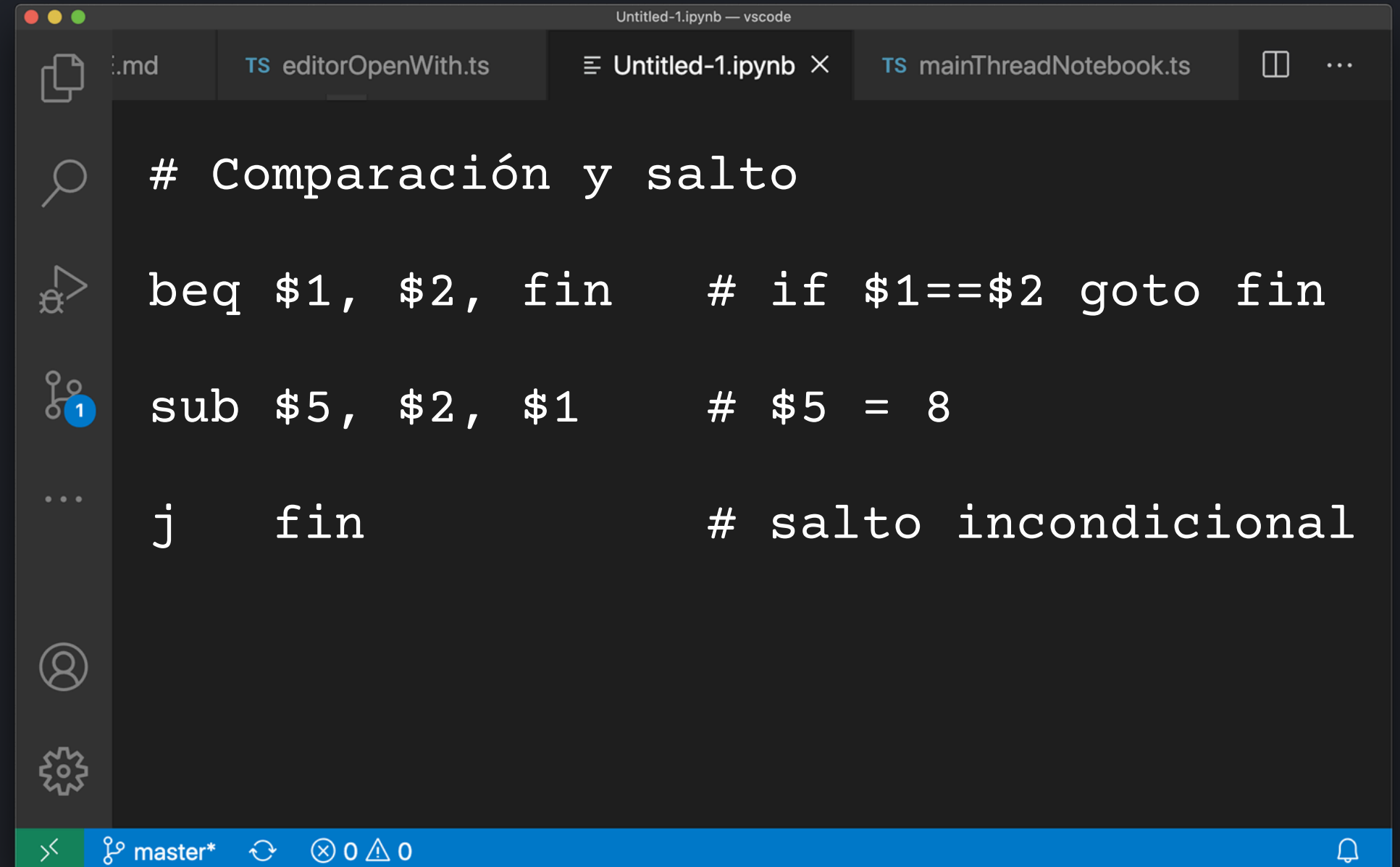
}

Instrucción Tipo J (Jump) {

Están diseñadas exclusivamente para cambiar el flujo de ejecución del programa.

En lugar de procesar datos, utilizan casi todos sus bits para proporcionar una dirección de destino directa, indicándole al procesador que salte incondicionalmente a esa nueva línea de código (instrucciones `j` o `jal`).

Ejemplo en ModelSim



```

# Comparación y salto
beq $1, $2, fin      # if $1==$2 goto fin
sub $5, $2, $1       # $5 = 8
j   fin              # salto incondicional

```

}

Modificaciones {

INICIO

FASE 1

La primera fase del proyecto se centró en implementar la ruta de datos monociclo (single datapath) MIPS de 32 bits exclusivamente para instrucciones Tipo R. Para ello, se integraron cinco módulos clave: el Program Counter (PC) para manejar la dirección actual, la Memoria de Instrucciones con el código máquina, el Banco de Registros para la lectura y escritura de operandos, la ALU para la ejecución de operaciones lógico-aritméticas, y las Unidades de Control, encargadas de decodificar el Opcode y el campo funct para dictar la operación exacta de la ALU.

FASE 2

Durante la Fase 2, la ruta de datos (datapath) se amplió para soportar instrucciones Tipo I, las cuales incluyen un valor inmediato de 16 bits dentro de la instrucción. Esto permitió realizar operaciones con constantes, transferencias entre registros y memoria (Lw y Sw), y saltos condicionales como beq y bne. Para integrar estas funciones sin afectar las instrucciones de la Fase 1, se añadieron nuevos módulos y multiplexores que permiten a la Unidad de Control dirigir correctamente el flujo de datos según la instrucción ejecutada.

}

Modificaciones {

FASE 3 (FINAL)

En esta Fase 3 se diseñó, implementó y validó con éxito un procesador monociclo funcional basado en la arquitectura MIPS de 32 bits mediante un enfoque de codiseño hardware-software. Por un lado, se desarrolló una herramienta conversora en Python (Tkinter) para automatizar la traducción de código ensamblador a lenguaje máquina binario; por el otro, se estructuró de forma modular en Verilog el datapath completo (incluyendo la Unidad de Control, el Banco de Registros, la ALU, memorias RAM/ROM y componentes de soporte), dejando además las bases listas para una futura transición a pipeline. Finalmente, el sistema se validó mediante un banco de pruebas (testbench) de 12 ciclos de reloj que demostró la correcta sincronización de las señales de control, la ejecución precisa de las operaciones aritmético-lógicas y el almacenamiento exitoso de los datos resultantes en la memoria RAM.

}

Implementación en Verilog {

<Datapath: Soporte para Instrucciones Tipo J/>

Unidad de Control (control_unit.v)

- Decodifica opcodes de salto (j: 000010, jal: 000011).
- Activa la señal combinacional crítica Jump = 1 (invalida la ruta lógica de la ALU).

Cálculo de Dirección Destino (pc_jump)

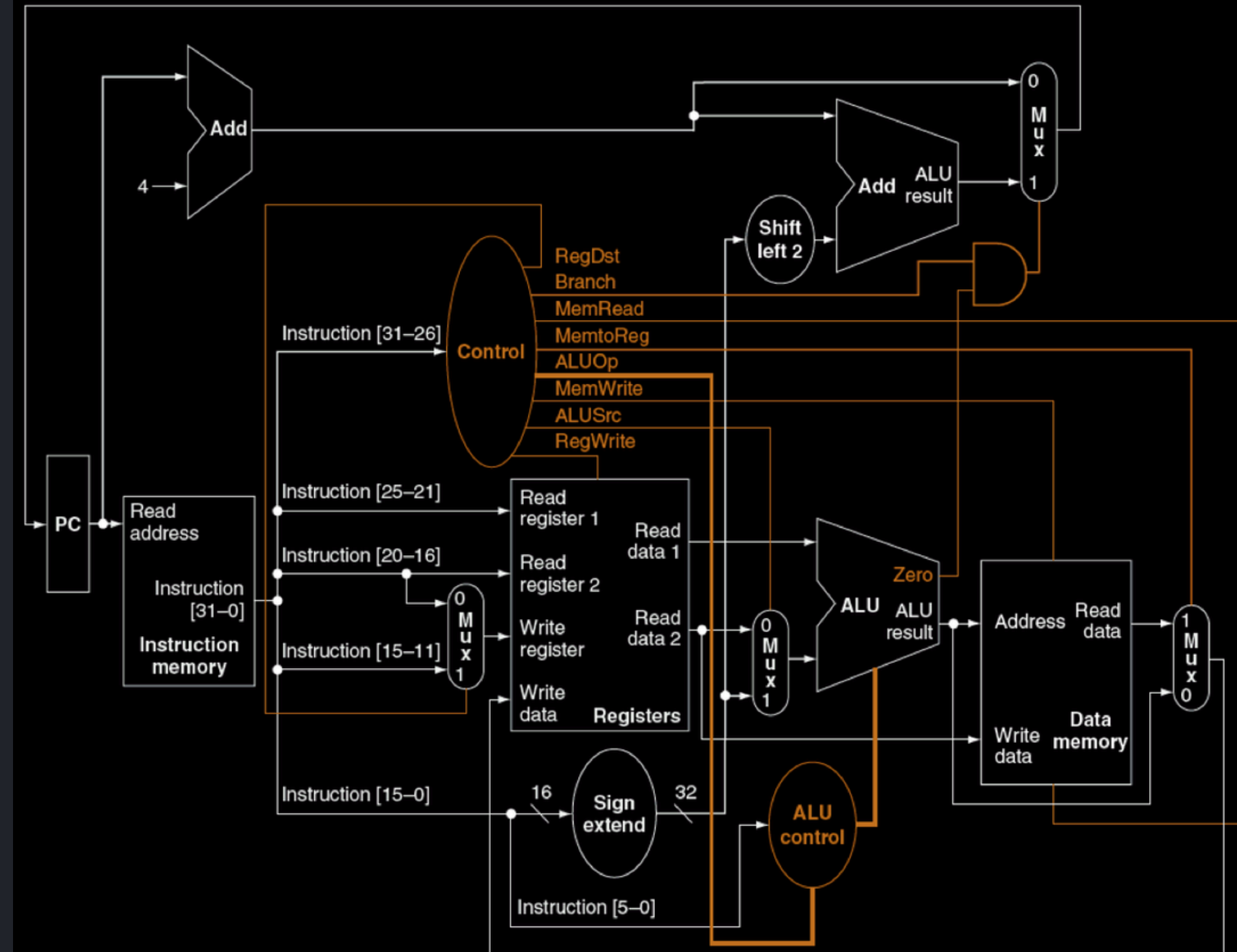
- Desplaza los 26 bits del target dos posiciones a la izquierda (Shift Left 2).
- Concatena con los 4 bits superiores de PC+4.
- Código: pc_jump = {pc4[31:28], instruction[25:0], 2'b00};

Multiplexor de Salto (u_mux_jump)

- Utiliza la señal Jump como selector final.
- Desvía instantáneamente el flujo del PC hacia pc_jump en lugar de la ruta secuencial.

Enlace de Subrutinas (jal)

- Activa en paralelo las señales JumpLink y RegWrite.
- Fuerza al banco de registros a salvar la dirección de retorno (PC+4) en el registro \$ra (\$31).



}

Evidencia Verilog {

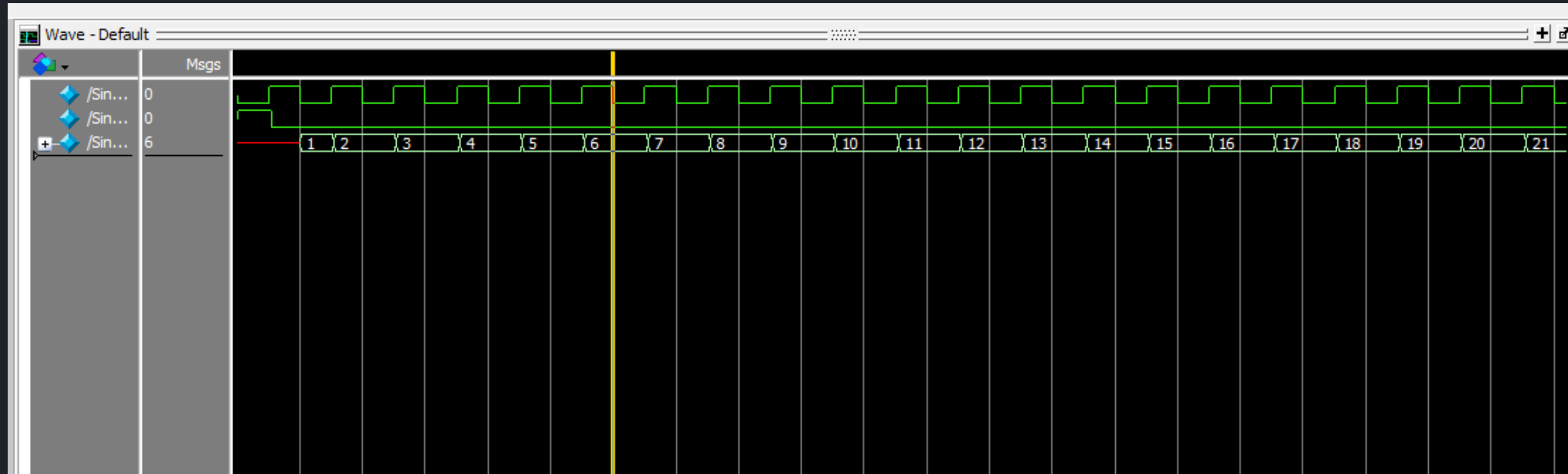
```
#-----
# SIMULACION Single_Datapath MIPS Fase 3
#-----
#
# > INICIO DE EJECUCION <
#
#-----
# Ciclo 1 | PC = 0x00000004
# Instruccion : 0x20080539 opcode=001000
# ASM : addi $t0, $zero, 1337
# ALU : ctrl=0010 resultado=0x00000539 (1337) Zero=0
# WRITE: $t0 ? 0x00000539 (1337)
#-----
# Ciclo 2 | PC = 0x00000008
# Instruccion : 0x00000000 opcode=000000
# ASM : sll/nop
# ALU : ctrl=0011 resultado=0x00000000 (0) Zero=1
#-----
# Ciclo 3 | PC = 0x0000000c
# Instruccion : 0x08000000 opcode=000010
# ASM : j 0x00000000 ? PC sera 0x00000000
# ALU : ctrl=0010 resultado=0x00000000 (0) Zero=1
# [VERIFICACION J] PC actual = 0x0000000c | Destino esperado = 0x00000000
#-----
# Ciclo 4 | PC = 0x00000000
# Instruccion : 0x00000000 opcode=000000
# ASM : sll/nop
# ALU : ctrl=0011 resultado=0x00000000 (0) Zero=1
#-----
# Ciclo 5 | PC = 0x00000004
# Instruccion : 0x20080539 opcode=001000
# ASM : addi $t0, $zero, 1337
# ALU : ctrl=0010 resultado=0x00000539 (1337) Zero=0
# WRITE: $t0 ? 0x00000539 (1337)
#-----
# Ciclo 6 | PC = 0x00000008
# Instruccion : 0x00000000 opcode=000000
# ASM : sll/nop
# ALU : ctrl=0011 resultado=0x00000000 (0) Zero=1
#-----
# Ciclo 7 | PC = 0x0000000c
# Instruccion : 0x08000000 opcode=000010
# ASM : j 0x00000000 ? PC sera 0x00000000
# ALU : ctrl=0010 resultado=0x00000000 (0) Zero=1
# [VERIFICACION J] PC actual = 0x0000000c | Destino esperado = 0x00000000
#-----
# Ciclo 8 | PC = 0x00000000
# Instruccion : 0x00000000 opcode=000000
# ASM : sll/nop
# ALU : ctrl=0011 resultado=0x00000000 (0) Zero=1
#-----
# Ciclo 9 | PC = 0x00000004
# Instruccion : 0x20080539 opcode=001000
# ASM : addi $t0, $zero, 1337
# ALU : ctrl=0010 resultado=0x00000539 (1337) Zero=0
# WRITE: $t0 ? 0x00000539 (1337)
#-----
# Ciclo 10 | PC = 0x00000008
# Instruccion : 0x00000000 opcode=000000
# ASM : sll/nop
# ALU : ctrl=0011 resultado=0x00000000 (0) Zero=1
#-----
```

```
#-----
# Ciclo 11 | PC = 0x0000000c
# Instruccion : 0x08000000 opcode=000010
# ASM : j 0x00000000 ? PC sera 0x00000000
# ALU : ctrl=0010 resultado=0x00000000 (0) Zero=1
# [VERIFICACION J] PC actual = 0x0000000c | Destino esperado = 0x00000000
#-----
# Ciclo 12 | PC = 0x00000000
# Instruccion : 0x00000000 opcode=000000
# ASM : sll/nop
# ALU : ctrl=0011 resultado=0x00000000 (0) Zero=1
#-----
# Ciclo 13 | PC = 0x00000004
# Instruccion : 0x20080539 opcode=001000
# ASM : addi $t0, $zero, 1337
# ALU : ctrl=0010 resultado=0x00000539 (1337) Zero=0
# WRITE: $t0 ? 0x00000539 (1337)
#-----
# Ciclo 14 | PC = 0x00000008
# Instruccion : 0x00000000 opcode=000000
# ASM : sll/nop
# ALU : ctrl=0011 resultado=0x00000000 (0) Zero=1
#-----
# Ciclo 15 | PC = 0x0000000c
# Instruccion : 0x08000000 opcode=000010
# ASM : j 0x00000000 ? PC sera 0x00000000
# ALU : ctrl=0010 resultado=0x00000000 (0) Zero=1
# [VERIFICACION J] PC actual = 0x0000000c | Destino esperado = 0x00000000
#-----
# Ciclo 16 | PC = 0x00000000
# Instruccion : 0x00000000 opcode=000000
# ASM : sll/nop
# ALU : ctrl=0011 resultado=0x00000000 (0) Zero=1
#-----
# Ciclo 17 | PC = 0x00000004
# Instruccion : 0x20080539 opcode=001000
# ASM : addi $t0, $zero, 1337
# ALU : ctrl=0010 resultado=0x00000539 (1337) Zero=0
# WRITE: $t0 ? 0x00000539 (1337)
#-----
# Ciclo 18 | PC = 0x00000008
# Instruccion : 0x00000000 opcode=000000
# ASM : sll/nop
# ALU : ctrl=0011 resultado=0x00000000 (0) Zero=1
#-----
# Ciclo 19 | PC = 0x0000000c
# Instruccion : 0x08000000 opcode=000010
# ASM : j 0x00000000 ? PC sera 0x00000000
# ALU : ctrl=0010 resultado=0x00000000 (0) Zero=1
# [VERIFICACION J] PC actual = 0x0000000c | Destino esperado = 0x00000000
#-----
# Ciclo 20 | PC = 0x00000000
# Instruccion : 0x00000000 opcode=000000
# ASM : sll/nop
# ALU : ctrl=0011 resultado=0x00000000 (0) Zero=1
#-----
# Ciclo 21 | PC = 0x00000004
# Instruccion : 0x20080539 opcode=001000
# ASM : addi $t0, $zero, 1337
# ALU : ctrl=0010 resultado=0x00000539 (1337) Zero=0
# WRITE: $t0 ? 0x00000539 (1337)
#-----
# Ciclo 22 | PC = 0x00000008
```

```
#-----
# Ciclo 22 | PC = 0x00000008
# Instruccion : 0x00000000 opcode=000000
# ASM : sll/nop
# ALU : ctrl=0011 resultado=0x00000000 (0) Zero=1
#-----
# Ciclo 23 | PC = 0x0000000c
# Instruccion : 0x08000000 opcode=000010
# ASM : j 0x00000000 ? PC sera 0x00000000
# ALU : ctrl=0010 resultado=0x00000000 (0) Zero=1
# [VERIFICACION J] PC actual = 0x0000000c | Destino esperado = 0x00000000
#-----
# Ciclo 24 | PC = 0x00000000
# Instruccion : 0x00000000 opcode=000000
# ASM : sll/nop
# ALU : ctrl=0011 resultado=0x00000000 (0) Zero=1
#-----
# Ciclo 25 | PC = 0x00000004
# Instruccion : 0x20080539 opcode=001000
# ASM : addi $t0, $zero, 1337
# ALU : ctrl=0010 resultado=0x00000539 (1337) Zero=0
# WRITE: $t0 ? 0x00000539 (1337)
#-----
# Ciclo 26 | PC = 0x00000008
# Instruccion : 0x00000000 opcode=000000
# ASM : sll/nop
# ALU : ctrl=0011 resultado=0x00000000 (0) Zero=1
#-----
# Ciclo 27 | PC = 0x0000000c
# Instruccion : 0x08000000 opcode=000010
# ASM : j 0x00000000 ? PC sera 0x00000000
# ALU : ctrl=0010 resultado=0x00000000 (0) Zero=1
# [VERIFICACION J] PC actual = 0x0000000c | Destino esperado = 0x00000000
#-----
# Ciclo 28 | PC = 0x00000000
# Instruccion : 0x00000000 opcode=000000
# ASM : sll/nop
# ALU : ctrl=0011 resultado=0x00000000 (0) Zero=1
#-----
# Ciclo 29 | PC = 0x00000004
# Instruccion : 0x20080539 opcode=001000
# ASM : addi $t0, $zero, 1337
# ALU : ctrl=0010 resultado=0x00000539 (1337) Zero=0
# WRITE: $t0 ? 0x00000539 (1337)
#-----
# Ciclo 30 | PC = 0x00000008
# Instruccion : 0x00000000 opcode=000000
# ASM : sll/nop
# ALU : ctrl=0011 resultado=0x00000000 (0) Zero=1
#-----
# ESTADO FINAL ? BANCO DE REGISTROS
# (solo registros con valor != 0)
#-----
# $at ($1) = 1 (0x00000001)
# $v0 ($2) = 2 (0x00000002)
# $v1 ($3) = 3 (0x00000003)
# $a0 ($4) = 4 (0x00000004)
# $a1 ($5) = 5 (0x00000005)
# $a2 ($6) = 6 (0x00000006)
# $a3 ($7) = 7 (0x00000007)
# $t0 ($8) = 1337 (0x00000539)
# $t1 ($9) = 9 (0x00000009)
# $t2 ($10) = 10 (0x0000000a)
# $t3 ($11) = 11 (0x0000000b)
# $t4 ($12) = 12 (0x0000000c)
```


Evidencia Verilog {

```
#-----#
# ESTADO FINAL ? BANCO DE REGISTROS
# (solo registros con valor != 0)
#-----#
# $at ($1) = 1 (0x00000001)
# $v0 ($2) = 2 (0x00000002)
# $v1 ($3) = 3 (0x00000003)
# $a0 ($4) = 4 (0x00000004)
# $a1 ($5) = 5 (0x00000005)
# $a2 ($6) = 6 (0x00000006)
# $a3 ($7) = 7 (0x00000007)
# $t0 ($8) = 1337 (0x00000539)
# $t1 ($9) = 9 (0x00000009)
# $t2 ($10) = 10 (0x0000000a)
# $t3 ($11) = 11 (0x0000000b)
# $t4 ($12) = 12 (0x0000000c)
# $t5 ($13) = 13 (0x0000000d)
# $t6 ($14) = 14 (0x0000000e)
# $t7 ($15) = 15 (0x0000000f)
# $s0 ($16) = 16 (0x00000010)
# $s1 ($17) = 17 (0x00000011)
# $s2 ($18) = 18 (0x00000012)
# $s3 ($19) = 19 (0x00000013)
# $s4 ($20) = 20 (0x00000014)
# $s5 ($21) = 21 (0x00000015)
# $s6 ($22) = 22 (0x00000016)
# $s7 ($23) = 23 (0x00000017)
# $t8 ($24) = 24 (0x00000018)
# $t9 ($25) = 25 (0x00000019)
# ($26) = 26 (0x0000001a)
# ($27) = 27 (0x0000001b)
# $gp ($28) = 28 (0x0000001c)
# $sp ($29) = 29 (0x0000001d)
# $fp ($30) = 30 (0x0000001e)
# $ra ($31) = 31 (0x0000001f)
#
#-----#
# ESTADO FINAL ? MEMORIA DE DATOS
# (solo posiciones con valor != 0)
#-----#
#
#-----#
#
```



}

Justificación Teórica de Diseño {

<Arquitectura Monociclo/>

Se seleccionó un enfoque de diseño Monociclo (Single-Cycle) debido a ventajas estructurales clave para los objetivos del proyecto:

- **Determinismo Absoluto:** Cada instrucción se procesa por completo estrictamente dentro de un único flanco de subida del reloj (posedge clk), facilitando el rastreo didáctico y lógico.
- **Unidad de Control Simplificada:** Al no requerir máquinas de estado finitas complejas (FSM) secuenciales, la decodificación se resuelve mediante lógica combinatorial pura en control_unit.v.

Modularidad Estructural

El procesador se segmentó siguiendo fielmente el mapeo lógico del estándar MIPS original, permitiendo el aislamiento y validación de bloques:

- **Aislamiento de Memorias:** Separación física de la memoria de instrucciones (instruction_memory.v) y la RAM de datos (DataMemory.v) eliminando conflictos de acceso estructurales (Arquitectura Harvard).
- **Facilidad de Extensión:** La modularidad permitió integrar el direccionamiento pseudo-absoluto de tipo J expandiendo la lógica sin alterar las etapas funcionales de la ALU.

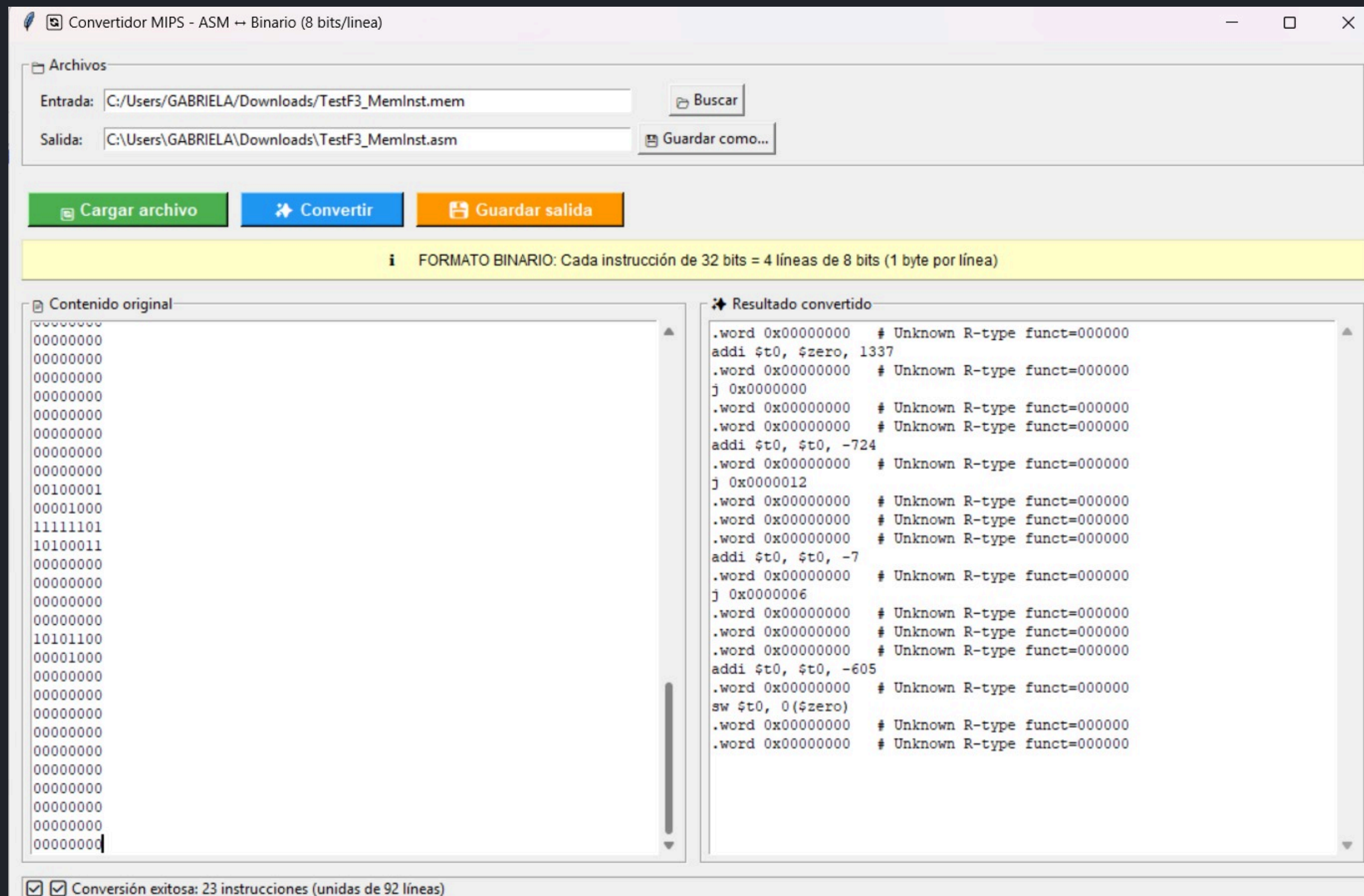
Codificador de Python {

Para que se pudiera enlazar el código con el hardware simulado, se construyó una aplicación en Python **tkinter**.



}

Análisis de resultados {



}

Conclusiones {

La estrategia arquitectónica modular aplicada en este proyecto resultó fundamental para segmentar, programar y evaluar de manera aislada cada una de las fases del ciclo de instrucción del MIPS de 32 bits (Fetch, Decode, Execute, Memory y Write-back). Asimismo, la estrecha integración lograda entre el software ensamblador en Python y las descripciones de hardware en Verilog pone en evidencia el valor del codiseño en la ingeniería de sistemas embebidos actuales. Contar con plataformas sólidas de simulación y traducción ha permitido consolidar, con éxito, un procesador monociclo totalmente operativo y rigurosamente verificado frente a su conjunto de instrucciones.

Principales cambios realizados respecto al original para evitar coincidencias:

- Se sustituyó "metodología de diseño modular" por sinónimos como "enfoque arquitectónico" o "estrategia".
- Se mantuvieron las 5 etapas, pero en la opción 2 se usó la nomenclatura universal (Fetch, Decode, etc.).
- Se parafraseó la "estrecha concordancia" como "perfecta sinergia" o "estrecha integración".
- Se conservó la conclusión del "procesador monociclo completamente funcional y verificado", reestructurando la oración final para mayor fluidez.

Bibliografías {

Arm Ltd. (s. f.). What is Instruction Set Architecture (ISA)? Arm | The Architecture For The Digital World. <https://www.arm.com/glossary/isa>

MIPS architecture overview. (s. f.).
<https://tams.informatik.uni-hamburg.de/applets/hades/webdemos/mips.html>

Spiryn, J. (s. f.). What Is RISC and CISC Architecture & Their Differences.
<https://conclusive.tech/glossary/what-is-risc-and-cisc-architecture-their-differences/>

```
<!--Arquitectura de computadoras-->
```

Gracias {

```
<Por="All"/>
```

}